

Runtime Control — Sysctl

How an operator inspects and tunes a running node without restarting it — a tree of named parameters exposed over a control socket.

SUBJECT hathor-core · Part II · the node, end to end

CHAPTER 41 · Part II · The Node

BRANCH study/hathor-core-studies

EDITION 2026 · v1

Sysctl · Runtime control · Command tree · Dotted-path parameters · Get/set · Unix socket · TCP · Live tuning · SIGUSR2

Table of Contents

Chapter 41 — Runtime Control: Sysctl	3
41.1 Localization	3
41.2 What it does, and why it exists	4
The problem: a daemon you cannot pause	4
The analogy: Linux sysctl	5
41.3 The concepts it rests on	6
41.4 The code, walked	6
41.4.1 A tiny toy first	6
41.4.2 The Sysctl base class — a node in the tree	7
41.4.3 Dotted-path addressing — walking down the tree	9
41.4.4 Get vs. set	9
41.4.5 The runner — from a text line to a call	10
41.4.6 The transport — a line-based socket protocol	11
41.4.7 Two real parameters, end to end	11
41.4.8 The per-subsystem nodes — a tour	13
41.5 How it plugs into the lifecycle	14
41.5.1 Assembly — the SysctlBuilder	14
41.5.2 Exposure — only if configured	15
41.5.3 The signal path — SIGUSR2	16
41.5.4 Putting it together	17
Recap	17

Chapter 41 — Runtime Control: Sysctl

What you'll learn in this chapter

- Why a long-running daemon needs a way to inspect and tune itself while running, and why "stop, edit a config file, restart" is not good enough for a mainnet node.
- The mental model behind `hathor/sysctl/`: a **tree of named parameters**, each with an optional **getter** and **setter**, addressed by a **dotted path** like `p2p.max_enabled_sync` — directly modelled on Linux's own `sysctl`.
- How the base `Sysctl` class stores that tree, how the **runner** parses a one-line `path / path=value` command, and how `get` and `set` are dispatched down the tree.
- How the tree is **wired to the live subsystems** (the P2P manager, storage, feature activation, ...) by the `SysctlBuilder`, and how it is exposed to the outside world — over a Unix socket or TCP, or through the `SIGUSR2` signal path.
- A few **real parameters** walked end to end, so you can read and write a knob on a running node yourself.

A full node is a daemon¹: you start it once and it runs for weeks. Over those weeks the operator's needs change. A peer is misbehaving and you want to cut it off. Sync is hammering a slow disk and you want fewer peers syncing at once. You suspect a memory leak and want to start the profiler for sixty seconds. None of these are worth a restart — a mainnet node restart means dropping every connection, re-reading the whole database, and being offline for minutes while it re-initializes. You need a side door: a way to reach into the running process, read a value, and change it, then close the door and leave the node running.

`hathor/sysctl/` is that side door. This chapter is short — the package is small (about seventeen files, most of them tiny) — but the idea is worth holding clearly, because it recurs: a **uniform tree of named, addressable parameters**, decoupled from the subsystems it controls and from the wire that carries the commands.

41.1 Localization

`hathor/sysctl/` sits in the infrastructure group of the codebase (Chapter 0, §0.4) — alongside the builder, the reactor abstraction, and the observability tools. It is operator-facing plumbing, not part of the ledger's data model.

```

hathor-core/
├── hathor/
│   ├── manager.py           ← HathorManager: the subsystems sysctl reads/tunes (Ch 29)
│   ├── builder/
│   │   └── sysctl_builder.py ← assembles the tree, wiring nodes to live subsystems (Ch
│   └── sysctl/              ◀ YOU ARE HERE
│       ├── sysctl.py        ← Sysctl base class: a node in the command tree
│       ├── runner.py        ← SysctlRunner: parse one line, dispatch get/set
│       ├── protocol.py      ← SysctlProtocol: the line-based socket protocol
│       ├── factory.py       ← SysctlFactory: builds a protocol per connection
│       ├── exception.py     ← the sysctl error hierarchy
│       ├── init_file_loader.py ← apply a file of commands at startup
│       ├── core/manager.py  ← HathorManagerSysctl (profiler, debugger, ipython)
│       ├── p2p/manager.py   ← ConnectionsManagerSysctl (sync knobs, kill peers)
│       ├── storage/manager.py ← StorageSysctl (rocksdb flush/stats)
│       ├── feature_activation/... ← FeatureActivationSysctl (signalling support)
│       └── websocket/manager.py ← WebsocketManagerSysctl (subscription limits)

```

Context. `hathor/sysctl/` is the runtime-control surface of the node. It does not own any state of its own; it is a thin, uniform shell wrapped around the real subsystems — the connections manager, the RocksDB storage, the bit-signalling service — letting an operator query and adjust them while the node keeps running. Everything it can read or change is a value that already lives in one of those subsystems (Chapter 29). The package's whole job is to give those values **stable names**, group them into a tree, and ferry one-line commands in and answers out.

41.2 What it does, and why it exists

The problem: a daemon you cannot pause

Most programs you have written run, do their job, and exit. You configure them by editing a file or passing flags before they start. If you got it wrong, you fix the file and run again. That loop is fine for a script that finishes in a second.

It is not fine for a node. A node is a server process that must stay up. Three things make "just restart it" a poor answer:

1. **Restarting is expensive.** On shutdown the node drops every peer connection; on startup it re-opens the database and rebuilds its in-memory view of the ledger (Chapter 29). For a synced mainnet node that is minutes of downtime, during which it sees and serves nothing.
2. **Some values are operational, not configuration.** "How many peers may sync at once" or "is the profiler running" are decisions you make in response

to what the node is doing right now. They are not the kind of thing you bake into a config file once and forget.

3. **You sometimes need to read internal state, not change it.** "Which sync versions are available?" "How big are the RocksDB memtables?" There is no config-file question for that — it is a live measurement.

So the node needs an interface that is alive while the node is alive: a way to ask "what is X right now?" and to say "set X to Y, now," without a restart and without recompiling anything.

The analogy: Linux `sysctl`

This problem is old, and the operating system you are reading this on already solved it. Linux exposes hundreds of kernel knobs through a tool called `sysctl`. Each knob has a **name** that looks like a path, with dots (or slashes) separating levels:

```
$ sysctl net.ipv4.tcp_syncookies          # read one knob
net.ipv4.tcp_syncookies = 1

$ sysctl -w net.core.somaxconn=1024      # write one knob
net.core.somaxconn = 1024
```

The kernel is running. You are changing its behaviour live. The names form a **tree**: `net` contains `ipv4` which contains `tcp_syncookies`. You address a single leaf by spelling out the full path.

Hathor copies this design wholesale — the package is even named `sysctl` — and applies it to the node instead of the kernel. The node exposes knobs like `p2p.max_enabled_sync` and `core.profiler.start`; you read one with `p2p.max_enabled_sync` and write one with `p2p.max_enabled_sync=5`. The shape is identical; only the subject changed from "the kernel" to "the running full node."

Recap — daemon (full treatment in Ch. 0). A daemon is a program that runs continuously in the background rather than finishing and exiting — a server, a database, a node. Because it is meant to stay up indefinitely, it needs ways to be observed and adjusted while running. Sysctl is one of those ways. → see Chapter 0, §0.1.

The conceptual win is **decoupling**, and it is worth naming because the same idea shows up across the codebase. The thing that holds a value (the connections manager) does not know it is reachable by sysctl. The thing that names the value (a small wrapper class) does not know how the command arrived — over a socket, from a file, or via a signal. And the thing that carries the command (the socket protocol) does not know what the value means. Three layers, each ignorant of the others' internals, joined only by narrow contracts. That is what lets the same parameter tree be driven three different ways (socket, init file, signal) without any of the parameter code changing.

41.3 The concepts it rests on

Sysctl is almost pure plumbing, so it leans on machinery introduced earlier. Three recaps, then the code.

Recap — the subsystems it controls (full treatment in Ch. 29). Everything sysctl reads or writes lives in a real subsystem assembled at boot. The `HathorManager` is the central coordinator that holds the others — the P2P `ConnectionsManager`, the RocksDB storage, the `BitSignalingService`, the admin WebSocket factory. Sysctl does not duplicate any of their state; it holds a reference to each and calls methods or reads attributes on it. When you understand a sysctl parameter, you are really understanding a method on one of those subsystems. → full treatment in Chapter 29.

Recap — the reactor it runs on (full treatment in Ch. 16 & 23). The node is built on **Twisted**, whose centre is the **reactor**: a single event loop that waits for events and calls your code in response. The sysctl socket is just another thing the reactor watches: when a byte arrives on the control socket, the reactor wakes up the sysctl protocol object. The socket is opened with a Twisted `endpoint` and served by a Twisted `factory` — both standard Twisted pieces you met in Chapter 16. → full treatment in Chapters 16 and 23.

Recap — command dispatch (full treatment in Ch. 4). Dispatch means: given a name, look up and call the right piece of code. The CLI does it for subcommands; sysctl does it for parameter paths. The runner takes a string like `p2p.max_enabled_sync`, walks a dictionary-backed tree to find the getter registered under that name, and calls it. There is no giant `if/elif` chain — just table lookups, the same pattern as the CLI dispatcher. → full treatment in Chapter 4.

41.4 The code, walked

41.4.1 A tiny toy first

Before the real class, build the idea from scratch with a neutral example. We want an object that holds a few named parameters, each of which you can get and/or set by name. A parameter is really just a pair of functions: a function that returns the current value, and a function that accepts a new one. Either may be missing (a read-only sensor has no setter; a fire-once button has no getter).

```

class ParamNode:
    def __init__(self):
        self._params = {} # name -> (getter, setter)

    def register(self, name, getter, setter):
        self._params[name] = (getter, setter)

    def get(self, name):
        getter, _ = self._params[name]
        if getter is None:
            raise PermissionError(f"{name} is write-only")
        return getter()

    def set(self, name, value):
        _, setter = self._params[name]
        if setter is None:
            raise PermissionError(f"{name} is read-only")
        setter(value)

```

Now wrap something real-ish — a thermostat — and expose one read-write knob and one read-only sensor:

```

class Thermostat:
    def __init__(self):
        self.target = 20.0
        self.measured = 18.7

knobs = ParamNode()
t = Thermostat()
knobs.register("target",
               getter=lambda: t.target,
               setter=lambda v: setattr(t, "target", v))
knobs.register("measured",
               getter=lambda: t.measured,
               setter=None) # read-only

knobs.get("target") # -> 20.0
knobs.set("target", 22) # changes the live thermostat
knobs.set("measured", 5) # -> PermissionError: read-only

```

Three things to carry forward. First, a parameter is a getter/setter pair, and a `None` in either slot means "that direction is forbidden." Second, the `ParamNode` holds no thermostat state of its own — it holds references to functions that read and write the live `Thermostat`. Third, dispatch is a dictionary lookup on the name. Hathor's `Sysctl` is exactly this, plus two refinements: the parameters form a **tree** (nodes can hold child nodes), so names become dotted paths; and a separate **runner** turns a text line into a `get / set` call. We meet both next.

41.4.2 The `sysctl` base class — a node in the tree

The real base class lives in `hathor/sysctl/sysctl.py`. A getter is a zero-argument callable returning anything; a setter takes one or more arguments and returns nothing (`sysctl.py:22`):

```

Getter = Callable[[], Any]
Setter = Callable[..., None]

```

```

class SysctlCommand(NamedTuple):
    getter: Optional[Getter]
    setter: Optional[Setter]

```

`SysctlCommand` (`sysctl.py:39`) is our getter/setter pair from the toy, given a name and made into a `NamedTuple`². Either field may be `None`, meaning that direction is not allowed.

The `Sysctl` class itself (`sysctl.py:44`) is one node in a tree. It holds two dictionaries: its child nodes, and its own leaf commands (`sysctl.py:47`):

```

class Sysctl:
    """A node in the sysctl tree."""

    def __init__(self) -> None:
        self._children: dict[str, 'Sysctl'] = {}
        self._commands: dict[str, SysctlCommand] = {}
        self.log = logger.new()

```

This is the crux of the tree shape. `_children` maps a path segment (like `'p2p'`) to another `Sysctl` node; `_commands` maps a name (like `'max_enabled_sync'`) to a getter/setter pair. A node can hold both — children and its own commands — which is how the tree gets its levels.

Two methods build the tree. `put_child` attaches a sub-node under a name (`sysctl.py:52`); `register` adds a leaf parameter (`sysctl.py:57`):

```

def register(self, path: str, getter: Optional[Getter], setter: Optional[Setter]) -> None:
    """Register a new parameter for sysctl."""
    assert path not in self._commands
    if setter is not None:
        setter = validate_call(setter)
    self._commands[path] = SysctlCommand(getter=getter, setter=setter)

```

Note the line `setter = validate_call(setter)`. `validate_call` is from **Pydantic** (Chapter 18): it wraps the setter so that the arguments are coerced and validated against the setter's type hints before the setter runs. This matters because the value is about to arrive as text from a socket. If a setter is declared `def set_max_enabled_sync(self, value: int)`, Pydantic guarantees `value` really is an `int` by the time the body runs — a malformed command is rejected at the boundary, not deep inside a subsystem. (This is the "parse, don't validate" idea from Chapter 18, applied to the control surface.)

41.4.3 Dotted-path addressing — walking down the tree

Now the lookup. Given a path like `p2p.max_enabled_sync`, how does the root node find the right leaf? The answer is a small recursion in `get_command` (`sysctl.py:67`):

```
def get_command(self, path: str) -> SysctlCommand:
    """Find and return the sysctl of the provided path."""
    if path in self._commands:
        return self._commands[path]
    for key, child in self._children.items():
        if not path.startswith(f'{key}.'):
            continue
        tail = path[len(key) + 1:]
        return child.get_command(tail)
    raise SysctlEntryNotFound(path)
```

Read it on the example `p2p.max_enabled_sync`, starting at the root node:

1. Is `'p2p.max_enabled_sync'` a leaf command directly on the root? No.
2. Walk the root's children. The child key `'p2p'` matches, because the path starts with `'p2p.'`. Strip that prefix and the dot: `tail = 'max_enabled_sync'`.
3. Recurse into the `p2p` child node with `'max_enabled_sync'`. There, step 1 finds it directly in `_commands`. Return that `SysctlCommand`.

If no leaf and no child prefix match, the node raises `SysctlEntryNotFound` (`exception.py:19`). The dot is the level separator, exactly as in Linux `sysctl`. Note a subtlety: a single node can register a dotted command name directly, like `'rate_limit.global.send_tips'` in the P2P node — that whole string is one leaf key in `_commands`, not three tree levels. So a path can have dots that are tree boundaries and dots that are just part of a leaf's name; `get_command` tries the direct-leaf match first, which makes that work.

41.4.4 Get vs. set

With the lookup in hand, reading and writing are thin. `get_getter` / `get_setter` (`sysctl.py:78`, `:85`) fetch the right half of the pair and enforce direction — asking to read a write-only entry raises `SysctlWriteOnlyEntry`, and vice versa:

```
def get_getter(self, path: str) -> Getter:
    cmd = self.get_command(path)
    if cmd.getter is None:
        raise SysctlWriteOnlyEntry(path)
    return cmd.getter
```

`get(path)` (`sysctl.py:92`) then calls the getter and returns its value. Setting goes through the runner (next), but the underlying `unsafe_set` (`sysctl.py:97`) shows the shape — and one wrinkle:

```
def unsafe_set(self, path: str, value: Any) -> None:
    setter = self.get_setter(path)
    if isinstance(value, tuple):
        setter(*value)
    else:
        setter(value)
```

The wrinkle: some setters take more than one argument. The send-tips rate limiter is set with two numbers — a count and a window in seconds. So if the parsed value is a tuple, it is unpacked into the setter's positional arguments; otherwise it is passed as a single argument. Keep that in mind when we reach the rate-limit example. (The method is called `unsafe_set` because it bypasses the runner's safety checks; the docstring says to use a runner instead — which is what every real path does.)

Two more methods, `get_all` (`sysctl.py:112`) and `get_all_paths` (`sysctl.py:122`), walk the whole tree depth-first to dump every readable parameter (used by the `!backup` command) or every path name (used by `!help`). They are the recursive cousins of `get_command`, visiting children first, then own commands.

41.4.5 The runner — from a text line to a call

The tree knows how to look up and call. What turns the operator's text — `p2p.max_enabled_sync=5` — into one of those calls? The `SysctlRunner` (`runner.py:24`). It holds the root node and has one entry method, `run` (`runner.py:31`):

```
def run(self, line: str, *, require_signal_handler_safe: bool = False) -> bytes:
    if not line:
        raise SysctlRunnerException('line cannot be empty or None')
    head, separator, tail = self.get_line_parts(line)
    if separator == '=':
        return self._set(head, tail, require_signal_handler_safe=require_signal_handler_safe)
    else:
        return self._get(head)
```

The grammar is one line, and the presence of an `=` decides the verb. `get_line_parts` (`runner.py:68`) splits on the first `=` with `str.partition`, then strips whitespace. No `=` means a **get**: `head` is the path, and `_get` calls the getter and serializes the answer. An `=` means a **set**: `head` is the path, `tail` is the value text.

The value text is **JSON**. `_set` (`runner.py:44`) hands `tail` to `deserialize` (`runner.py:83`), which parses it as JSON — so you write `p2p.max_enabled_sync=5` (the `5` is JSON for the integer five), and a string value would be `core.something="hello"` with the quotes. The clever bit handles multi-argument setters: `deserialize` wraps the text in brackets and parses it as a JSON array, and if that yields more than one element it returns a **tuple** (`runner.py:88`). So `p2p.rate_limit.global.send_tips=100, 60` deserializes to the tuple `(100, 60)`, which — per the unpacking rule from §41.4.4 —

calls the setter with two arguments. On the way back out, getter results are turned to text by `serialize (runner.py:75)`: tuples become comma-joined JSON parts, everything else is plain JSON.

There is one more parameter on `run: require_signal_handler_safe`. Ignore it for now; it belongs to the signal path and we explain it in §41.5.

41.4.6 The transport — a line-based socket protocol

The runner is transport-agnostic: it eats a string and returns bytes. Something has to carry those strings. For network access that something is the `SysctlProtocol` (`protocol.py:31`), a Twisted `LineReceiver`:

```
class SysctlProtocol(LineReceiver):
    delimiter = b'\n'

    def __init__(self, runner: SysctlRunner) -> None:
        self.runner = runner

    def lineReceived(self, raw: bytes) -> None:
        ...
        feedback = self.runner.run(line)
        if feedback:
            self.sendLine(feedback)
```

A `LineReceiver`³ is a Twisted protocol that buffers incoming bytes and calls `lineReceived` once per complete line (here, lines end in `\n`). So the wire format is dead simple: send one command per line, get one line of answer (or an error line) back. This is what makes the control socket usable with bare tools — you can `nc` (netcat) into it and type commands by hand, or pipe a file of commands at it.

`lineReceived (protocol.py:37)` also recognizes a few meta-commands before handing the line to the runner: `!help [path]` prints available paths or the docstrings of one command's getter/setter (built by reflection with `inspect`, `protocol.py:111`), and `!backup` dumps every readable parameter and its current value via `get_all`. Errors are caught per type — `SysctlEntryNotFound`, `SysctlReadOnlyEntry`, Pydantic's `ValidationError`, and so on — and turned into a human-readable `[error] ...` line rather than crashing the connection (`protocol.py:55`).

One protocol instance is created per connection by the `SysctlFactory (factory.py:21)`, a Twisted `Factory` whose `buildProtocol` returns a fresh `SysctlProtocol` wired to the shared runner. Factory and protocol are the standard Twisted server pair from Chapter 16: the factory is the long-lived listener, a protocol is one conversation.

41.4.7 Two real parameters, end to end

Abstractions land when you trace a real one. Take two from the P2P node, `hathor/sysctl/p2p/manager.py`.

Read-write integer — `p2p.max_enabled_sync`. This caps how many peers may run the sync protocol at once. The `ConnectionsManagerSysctl` registers it in its constructor, pointing the getter and setter at its own methods (`p2p/manager.py:65`):

```
self.register(
    'max_enabled_sync',
    self.get_max_enabled_sync,
    self.set_max_enabled_sync,
)
```

The getter reads a live attribute off the connections manager (`p2p/manager.py:180`); the setter validates the input and pokes the same attribute, then forces a sync re-shuffle so the new cap takes effect immediately (`p2p/manager.py:185`):

```
def get_max_enabled_sync(self) -> int:
    """Return the maximum number of peers running sync simultaneously."""
    return self.connections.MAX_ENABLED_SYNC

@signal_handler_safe
def set_max_enabled_sync(self, value: int) -> None:
    """Change the maximum number of peers running sync simultaneously."""
    if value < 0:
        raise SysctlException('value must be >= 0')
    if value == self.connections.MAX_ENABLED_SYNC:
        return
    self.connections.MAX_ENABLED_SYNC = value
    self.connections._sync_rotate_if_needed(force=True)
```

Now the full path of `p2p.max_enabled_sync=5` over the socket:

```
operator types:  p2p.max_enabled_sync=5
|
v
SysctlProtocol.lineReceived → runner.run("p2p.max_enabled_sync=5")
|   sees '=', so this is a SET; tail "5" → deserialize → int 5
v
runner._set → root.get_setter("p2p.max_enabled_sync")
|   get_command walks: root → child "p2p" → leaf "max_enabled_sync"
|   (Pydantic-wrapped) setter called with value=5
v
ConnectionsManagerSysctl.set_max_enabled_sync(5)
|   validates >= 0, writes connections.MAX_ENABLED_SYNC = 5,
|   forces a sync rotate so it applies now
v
runner returns b'' → protocol sends nothing back (set has no output)
```

A read, `p2p.max_enabled_sync`, takes the same path minus the `=`: the runner sees no separator, calls the getter, JSON-serializes the returned `int`, and the protocol writes it back as one line.

Read-only list — `p2p.available_sync_versions`. Registered with a getter and **no** setter (`p2p/manager.py:95`):

```

self.register(
    'available_sync_versions',
    self.get_available_sync_versions,
    None,
)

```

Because the setter slot is `None`, a `p2p.available_sync_versions=...` write fails at `get_setter` with `SysctlReadOnlyEntry`, which the protocol reports as `[error]` cannot write to `p2p.available_sync_versions`. A read returns the list of supported sync versions (currently just `v2`; `sync-v1` is removed — Chapter 35). This is the "sensor" case from the toy: observable, not adjustable.

Multi-argument setter — `p2p.rate_limit.global.send_tips`. Worth one more look because it exercises the tuple path. Its setter takes two values (`p2p/manager.py:137`):

```

def set_global_send_tips_rate_limit(self, max_hits: int, window_seconds: float) -> None:
    """Change the global rate limiter for SEND_TIPS.
    The rate limiter is disabled when `window_seconds == 0`."""

```

Sending `p2p.rate_limit.global.send_tips=100, 60` makes `deserialize` produce the tuple `(100, 60)`, which the runner unpacks into `set_global_send_tips_rate_limit(100, 60)` — a rate limit of 100 messages per 60-second window. Setting the window to `0` turns the limiter off. This is why the getter/setter contract allows `Setter = Callable[..., None]` (variadic) rather than a single-argument function.

41.4.8 The per-subsystem nodes — a tour

Each subsystem gets its own `Sysctl` subclass, holding a reference to the live object it controls and registering that object's knobs in its constructor. They are all the same shape; here is what each exposes.

- **ConnectionsManagerSysctl** (`p2p/manager.py:60`) — the richest node. Sync tuning (`max_enabled_sync`, `enabled_sync_versions`, `sync_update_interval`, `force_sync_rotate`), the `send_tips` rate limiter, peer-management actions (`kill_connection` to drop one peer or `*` for all; `always_enable_sync` to pin peers; `reload_entrypoints_and_connections`), and hostname control.
- **HathorManagerSysctl** (`core/manager.py:24`) — debugging and profiling, mounted at `core.profiler.start` / `profiler.stop` / `profiler.status` (Chapter 42), plus live-debugger doors: `pudb.set_trace.*` drops into the `pudb` debugger on a chosen TTY, and `ipython.run.attach_tty` opens an IPython shell with `manager` and `tx_storage` pre-bound. These last ones pause the main loop and are flagged destructive in their docstrings — they exist for incident forensics, not routine use.
- **StorageSysctl** (`storage/manager.py:23`) — RocksDB operations: `rocksdb.flush` forces memtables to disk, `rocksdb.memtable_stats` and `rocksdb.wal_stats` report live storage sizes (Chapter 27). All under the `storage` prefix.

- **FeatureActivationSysctl** (`feature_activation/manager.py:20`) — mounted at `core.features` . Reads which features this node supports / is signalling for, and `add_support` / `remove_support` flip this node's signalling bits for a named feature (Chapter 38).
- **WebsocketManagerSysctl** (`websocket/manager.py:20`) — mounted at `ws` , present only if the admin WebSocket is enabled. Caps on subscribed addresses per connection (`max_subs_addrs_conn` , `max_subs_addrs_empty`), with `-1` meaning unlimited (Chapter 36).

The pattern is uniform: a subsystem node wraps a live object, names its knobs, and translates between text-friendly values and the object's real types. None of these classes hold state; they are naming-and-validation shells over the real subsystems.

41.5 How it plugs into the lifecycle

We have the tree, the runner, and the transport. Three questions remain: who builds the tree and wires it to the live subsystems, when is it exposed, and what is the `SIGUSR2` path for.

41.5.1 Assembly — the `SysctlBuilder`

The tree is not built by hand at every call site. The `SysctlBuilder` (`hathor/builder/sysctl_builder.py:26`) does it, given the `BuildArtifacts` bundle the main `Builder` produced (Chapter 24). Its `build` reads almost as a picture of the tree (`sysctl_builder.py:32`):

```
def build(self) -> Sysctl:
    root = Sysctl()

    core = HathorManagerSysctl(self.artifacts.manager)
    core.put_child('features', FeatureActivationSysctl(self.artifacts.bit_signaling_service))

    root.put_child('core', core)
    root.put_child('p2p', ConnectionsManagerSysctl(self.artifacts.p2p_manager))
    root.put_child('storage', StorageSysctl(self.artifacts.rocksdb_storage))

    ws_factory = self.artifacts.manager.websocket_factory
    if ws_factory is not None:
        root.put_child('ws', WebsocketManagerSysctl(ws_factory))

    return root
```

This is where the **wiring** happens. Each subsystem node is constructed with the real live object pulled from `BuildArtifacts` — the actual `HathorManager` , the actual `ConnectionsManager` , the actual RocksDB storage. That is what makes a `sysctl` `set` reach the running node rather than a copy: the getter/setter closures captured these references at build time. The tree's shape — `core` , `core.features` , `p2p` , `storage` ,

optional `ws` — is fixed right here. The `ws` node only appears if the WebSocket surface was enabled, so the parameter tree's exact contents depend on how the node was started.

41.5.2 Exposure — only if configured

A sysctl tree that no one can reach is harmless. The control surface is **off by default**; the operator opts in with a CLI flag. In `run_node.py`, after the node is prepared, the start path checks for `--sysctl` and, if present, calls `init_sysctl` (`run_node.py:544`):

```
self.prepare()
self.register_signal_handlers()
if self._args.sysctl:
    self.init_sysctl(self._args.sysctl, self._args.sysctl_init_file)
```

`init_sysctl` (`run_node.py:557`) builds the runner, optionally applies an init file, then opens the listening socket from a Twisted endpoint string (`run_node.py:569`):

```
runner = self.get_sysctl_runner()
if sysctl_init_file:
    init_file_loader = SysctlInitFileLoader(runner, sysctl_init_file)
    init_file_loader.load()
factory = SysctlFactory(runner)
endpoint = serverFromString(self.reactor, description)
endpoint.listen(factory)
```

The `--sysctl` value is a Twisted endpoint description, so the same code serves either transport depending on what you pass (`run_node.py:560`):

```
--sysctl unix:/path/sysctl.sock          # a Unix-domain socket (local only)
--sysctl unix:/path/sysctl.sock:mode=660 # ... with file permissions
--sysctl tcp:5000                        # a TCP port
--sysctl tcp:5000:interface=127.0.0.1    # ... bound to localhost
```

`serverFromString` parses that string into the right kind of listening endpoint, and `endpoint.listen(factory)` registers it with the reactor (Chapter 16). From then on, a connection to that socket gets a `SysctlProtocol` and can issue commands. The choice of transport is a security posture: a Unix socket with `mode=660` is reachable only by local processes the file permissions allow; a TCP port is reachable by anything that can route to it, so it is normally bound to `127.0.0.1`. The control surface has **no authentication of its own** — whoever can open the socket can tune the node — so the access control is the socket's, which is why the Unix-socket-with-permissions form is the safe default.

The optional `--sysctl-init-file` applies a file of commands once, at startup, through `SysctlInitFileLoader` (`init_file_loader.py:4`). It reads the file line by line and feeds each line to `runner.run` (`init_file_loader.py:12`) — the same runner the socket uses.

So the init file is "a batch of sysctl commands applied at boot," handy for setting non-default operational values (a custom sync cap, a rate limit) without a permanent config change. It reuses the entire runner machinery; there is no separate parser.

41.5.3 The signal path — SIGUSR2

There is a second way in, for when there is no socket open: a Unix **signal**⁴. At startup `register_signal_handlers` (`run_node.py:296`) installs a handler for `SIGUSR2` (where the OS provides it):

```
sigusr2 = getattr(signal, 'SIGUSR2', None)
if sigusr2 is not None:
    signal.signal(sigusr2, self.signal_usr2_handler)
```

When the process receives `SIGUSR2`, the handler calls `run_sysctl_from_signal` (`run_node.py:326`). This path is deliberately different from the socket. It **pauses the main loop**, creates a temporary named pipe (FIFO) at `SIGUSR2-<pid>.pipe` in the data directory, and blocks reading commands from that pipe; an operator writes commands into the pipe, the node executes them, then resumes (`run_node.py:345`):

```
filename = os.path.join(basedir, f'SIGUSR2-{os.getpid()}.pipe')
...
with temp_fifo(filename, tempdir):
    fp = open(filename, 'r')
    lines = fp.readlines()
    ...
    for cmd in lines:
        output = runner.run(cmd, require_signal_handler_safe=True)
```

Two things make this safe. First, this is the same runner over the same tree — no separate command set. Second, note `require_signal_handler_safe=True`. Because the signal handler can interrupt the node at any instruction — possibly mid-update of some data structure — only commands explicitly marked safe to run in that frozen, mid-flight state are allowed. That mark is the `@signal_handler_safe` decorator (`sysctl.py:30`), which sets an attribute on the method; the runner refuses any setter lacking it when called from the signal path (`runner.py:52`):

```
@signal_handler_safe
def set_max_enabled_sync(self, value: int) -> None:
    ...
```

So `set_max_enabled_sync` can be driven by `SIGUSR2` (it is decorated), but a setter that touches fragile state and is not decorated cannot. The socket path imposes no such restriction, because there the reactor delivers the command at a clean moment between events, not mid-instruction.

Recap — SIGUSR2 and the signal handler (full treatment in Ch. 29). A signal is a minimal asynchronous notification the OS delivers to a process. `SIGUSR1` and `SIGUSR2` are reserved for an application to define. Hathor uses `SIGUSR1` to reload peer entryptoints and `SIGUSR2` to open the FIFO-backed sysctl path described above. The handlers are installed in `run_node.py`, not in the manager. → the surrounding lifecycle is Chapter 29.

41.5.4 Putting it together

The whole lifecycle, in one line each:

```
build time   SysctlBuilder.build() constructs the tree, wiring each node to a
              live subsystem from BuildArtifacts                                (Ch 24)
start (opt)  --sysctl opens a Unix/TCP socket serving SysctlProtocol;
              --sysctl-init-file replays a batch of commands once
run (socket) a line in → runner parses get/set → tree dispatch → live subsystem
run (signal) SIGUSR2 → pause loop → read FIFO → runner (signal-safe only) → resume
```

At every step the parameter code is untouched: the same `Sysctl` tree and the same `SysctlRunner` serve the socket, the init file, and the signal pipe. That is the payoff of the three-layer decoupling from §41.2 — the knobs were defined once, and three different mechanisms drive them.

Recap

Parameter path	Subsystem (node class)	Get	Set	Notes
<code>p2p.max_enabled_sync</code>	<code>ConnectionsManagerSysctl</code>	yes	yes	cap on peers syncing at once; signal-safe
<code>p2p.available_sync_versions</code>	<code>ConnectionsManagerSysctl</code>	yes	—	read-only sensor (currently v2)
<code>p2p.rate_limit.global.send_tips</code>	<code>ConnectionsManagerSysctl</code>	yes	yes	two-arg setter (hits, window); <code>0</code> disables

Parameter path	Subsystem (node class)	Get	Set	Notes
p2p.kill_connection	ConnectionsManagerSysctl	—	yes	drop one peer, or * for all; signal-safe
core.profiler.start	HathorManagerSysctl	—	yes	start CPU profiler (Ch 42); signal-safe
core.profiler.status	HathorManagerSysctl	yes	—	(enabled, duration)
core.features.add_support	FeatureActivationSysctl	—	yes	flip this node's signalling bit (Ch 38)
storage.rocksdb.flush	StorageSysctl	—	yes	force memtables to disk (Ch 27); signal-safe
storage.rocksdb.wal_stats	StorageSysctl	yes	—	live WAL file sizes
ws.max_subs_addrs_conn	WebsocketManagerSysctl	yes	yes	only if WebSocket enabled (Ch 36); -1 = unlimited

The whole package is one idea applied with discipline: name every operationally-interesting value, arrange the names into a dotted tree, and decouple naming from the value's home subsystem and from the wire that carries the command. The base `Sysctl` is a tree node holding child nodes and getter/setter leaves; the `SysctlRunner` turns one text line into a get or a set; a Twisted `LineReceiver` protocol carries lines over a Unix socket or TCP; the `SysctlBuilder` wires each tree node to a live subsystem at boot; and `--sysctl`, `--sysctl-init-file`, and `SIGUSR2` are three independent doors onto the very same tree. With `sysctl` you can change a running node. The next chapter, **Chapter 42 (Observability)**, covers the other half of the operator's job — watching a running node: Prometheus metrics, the CPU profiler, and the healthcheck endpoint, the surfaces through which the node reports on itself rather than being adjusted.

-
-
1. A daemon is a program that runs continuously in the background rather than finishing and exiting — a server, a database, a node. Pronounced "demon." Full treatment in Chapter 0. ↩
 2. A `NamedTuple` is a tuple whose fields also have names, so you can write `cmd.getter` instead of `cmd[0]`. It is immutable and lightweight — handy for small fixed records like a getter/setter pair. See Chapter 1. ↩
 3. A `LineReceiver` is a Twisted protocol that buffers raw incoming bytes and calls your `lineReceived(line)` method once per complete line (split on a delimiter, here `\n`). It saves you from reassembling lines out of arbitrary network chunks. Twisted protocols are covered in Chapter 16. ↩
 4. A signal is a small asynchronous notification the operating system delivers to a process (e.g. `SIGINT` from Ctrl-C). `SIGUSR1` and `SIGUSR2` are left undefined by the OS for an application to use as it likes. A signal handler can interrupt the program at almost any point, which is why `sysctl` restricts what may run from the `SIGUSR2` path. ↩